

Results Brief: An AI Agent Autonomously Competing in Kaggle Tabular Competitions

This brief condenses the results of the entire summer project, for use in oral reports and quick review; the full methodology is in the Preface (PREFACE), and the per-competition and cross-competition data are in the Summary Report (SUMMARY_REPORT) and each competition report. All numbers are taken from reproducible raw logs and pass automatic consistency verification.

Contents	
1. One-Sentence Objective	1
2. What Was Done (Deliverables)	1
3. Core Method: Five-Stage Ablation	2
4. Main Results	2
5. Transferable Insights Learned Across Competitions	3
6. Alignment with ERA of Aygün et al. (Project Brief 4.1)	3
7. Against the Project Brief's Five Objectives	4
8. Next Steps	4

1. One-Sentence Objective

To evaluate whether a hybrid AI agent, "Claude Code (LLM agent) + AutoML tools," can **autonomously** work through the complete data-science pipeline of a Kaggle tabular competition (understanding the problem → EDA → CV design → features → modeling → ensembling → submission), and to use **controlled ablation** to quantify "how much more the score improves with each added autonomous capability." The method aligns with the core claims of the ERA system by Aygün et al. (2026, Nature).

2. What Was Done (Deliverables)

Deliverable	Content
kaggle-agent skill	Competition-workflow operating manual: the six stages EDA→CV→features→modeling→evaluation→submission
self-improvement skill	Linear self-iteration + cross-competition experience library (accumulated entries of what works / what doesn't)
Custom tree-search tool (3 versions)	Candidate tree + backtracking + budget phase machine, replacing linear single-path optimization

Deliverable	Content
External idea bank + injection hook	24 literature techniques with real provenance ([EXT]); the v4 harness pools it with the experience library and injects into the tree search
kaggle-report skill	Automatic report generation: fact extraction → numeric consistency verification → PDF with table of contents / page numbers
15 competitions + 3 overviews	Per-competition reports + preface + summary report + this brief, all generated and verified automatically

3. Core Method: Five-Stage Ablation

Solve the same competition multiple times, giving one additional capability each time, then compare the scores; this isolates each capability's contribution segment by segment:

- **Stage 1:** no-skill baseline (run generic AutoML directly)
- **Stage 2:** + kaggle-agent skill (structured six-stage workflow)
- **Stage 3:** + linear self-iteration (experience-library priors, Optuna, seed bagging)
- **Stage 4:** + tree search (candidate tree replaces the linear single path → corresponds to ERA's first pillar)
- **Stage 5:** + external-idea injection (literature idea bank → corresponds to ERA's second pillar; the injection mechanism was built and empirically tested—external injection yields no measurable systematic gain on the test competitions (an honest null), see § 8)

4. Main Results

(1) The incremental ladder holds in every competition: across 15 competitions (10 same-season S3 main benchmark + 5 cross-season S4–S6), Stages 1→4 are positive or flat in every competition, **with no competition regressing overall at any stage**—this is cross-competition evidence that "introducing autonomous capabilities really does help" (project-brief objective three: performance does not regress).

(2) The distribution of the gains is itself information: large improvements concentrate on competitions where "the data hides structural insight" (e.g., s3e20 +25.7%, s3e5 +19.2%), whereas competitions where the baseline is already near the ceiling generally have increments <1%. This shows that the agent's value lies in "finding and exploiting structure," not in indiscriminately stacking models.

(3) Real leaderboard validation: s3e16 is the only competition with a real Kaggle LB comparison; the tree-search version (Stage 4) improves over the linear version (Stage 2) **in the same direction on both the Public and Private boards**, and the CV↔LB gap is consistent across the two submissions—this is the method's first external (non-local) reliability evidence.

(4) Cross-season generalization: transplanting the four-stage recipe validated on S3 unchanged to five competitions in Seasons 4–6 (five different metrics: AUC, accuracy, RMSE, R2), the ladder **still holds in every competition**, only with converging magnitudes—the recipe generalizes to new seasons and new metrics.

5. Transferable Insights Learned Across Competitions

- **Experience-library priors must be checked competition by competition—the same prior can yield opposite conclusions:** the interaction-term prior was confirmed on s5e10/s6e2 but rejected on s6e1; class-imbalance weighting has different applicable boundaries across metrics. Priors must be transferred together with their conditions of applicability.
- **The metric decides post-processing, not the appearance of the target:** an integer target pairs with rounding (s3e16), an ordinal one with OptimizedRounder (s3e5); but although the s5e10 target falls on grid points, snapping to the grid actually hurts RMSE—discrete appearance $\neq$ should be discretized.
- **The winning shape of the tree search varies with the data:** in some competitions a mega-blend wins, in others a lineage blend wins, in others a single model wins—a fixed playbook would bet on the wrong shape, whereas the search finds the right one on the spot.
- **Reproducibility discipline:** every Stage 4 best solution in the cross-season competitions passes the bit-by-bit OOF reproducibility gate; and LightGBM's cross-process non-determinism was fixed, reaching bit-level reproducibility with $\max|dOOF|=0$ —this is the root of the ablation methodology's credibility.

6. Alignment with ERA of Aygün et al. (Project Brief 4.1)

ERA pillar	This project's counterpart	Status
First pillar: candidate tree replaces the linear single path	Stage 4 tree search (3-version tool)	Completed, validated on 15 competitions
Second pillar: external- knowledge injection	Stage 5 external idea bank + v4 injection hook + plateau injection	Idea bank of 24 entries + pooled deduplication + injection mechanism (idea_injection.py) built and empirically tested. Honest conclusion: the mechanism is feasible, but external injection yields no measurable systematic gain (null) on the test competitions—ideas that can be cleanly translated are already covered by the search, and useful ideas are already hand-written in (see phase_j_j3_findings)

7. Against the Project Brief's Five Objectives

Objective	Status
One: automatic report generation	kaggle-report skill, fully automatic output for all 15 competitions
Two: report quality (readable, no self-coined jargon)	preface extracts shared content, plain-language phrasing, table of contents / page numbers
Three: performance does not regress	15-competition ladder holds in every competition
Four: decisions are traceable	numeric-consistency verification gate + bit-by-bit OOF reproducibility gate
Five: reproducible and deployable	reproducibility reaches bit level; packaging/containerization in progress

8. Next Steps

- **Stage 5 (external injection) is wrapped up:** J-3 found that `tree['priors']` was write-only → it has been wired up (`idea_injection.py`, plateau-triggered, capped count) and empirically tested. Honest answer: **external injection yields no measurable systematic gain (null)** on the test competitions —ideas that can be cleanly translated (rank averaging) are already covered by the search, and useful ideas are already hand-written or hard to translate automatically. ERA's second pillar is positioned as a rigorous negative result of "mechanism feasible, marginal value null" (see docs/`phase_j_j3_findings.md`). If one wanted to explore further, the direction would be ideas that "require retraining/new features" (adversarial validation, etc.), but the consistent pattern points to diminishing returns.
- Delivery-layer wrap-up: skill packaging, usage instructions, containerization (the local machine is arm64, so package builds are a variable).
- Statistical-rigor upgrade: multi-seed reruns + confidence intervals, turning "Stage N > Stage N-1" from a point estimate into statistical significance.